FLA Project – Java Implementation

Shivanandana A

CB.SC.U4AIE24049

1. DFA Simulation

Simulate a given DFA and determine whether a given input string is accepted or rejected.

Code

```
import java.util.*;

public class DFASimulation {
    // delta[state][symbol], accepts strings ending in 1
    static final int[][] DELTA = {{0, 1}, {0, 1}};
    static final boolean[] ACCEPT = {false, true};

    static boolean accepts(String s) {
        int state = 0;
        for (char ch : s.toCharArray()) {
            if (ch != '0' && ch != '1') return false;
            state = DELTA[state][ch - '0'];
        }
        return ACCEPT[state];
    }

    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            System.out.print("Enter a binary string: ");
            String s = sc.nextLine().trim();
            System.out.println(accepts(s) ? "Accepted" : "Rejected");
        }
    }
}
```

Sample Output

```
Enter a binary string: 1011
Accepted
```

```
Enter a binary string: 110
Rejected
```

2. NFA to DFA Conversion

Convert a given NFA into an equivalent DFA using the subset construction method.

Code

```
import java.util.*;

public class NFAToDFA {
    // nfa[state][symbol] = destination states
    static final int[][][] NFA = {
        {{0, 1}, {0}},
        {{},     {1}}
    };
    static final Set<Integer> NFA_ACCEPT = Set.of(1);
```

```
    static Set<Integer> move(Set<Integer> from, int sym) {
        Set<Integer> to = new TreeSet<>();
        for (int s : from)
            for (int d : NFA[s][sym]) to.add(d);
        return to;
    }

    public static void main(String[] args) {
        Map<Set<Integer>, Integer> id = new LinkedHashMap<>();
        List<int[]> trans = new ArrayList<>();
        Queue<Set<Integer>> queue = new ArrayDeque<>();

        Set<Integer> start = new TreeSet<>(Set.of(0));
        id.put(start, 0);
        queue.add(start);

        while (!queue.isEmpty()) {
            Set<Integer> cur = queue.poll();
            int[] row = new int[2];
            for (int sym = 0; sym < 2; sym++) {
                Set<Integer> next = move(cur, sym);
                if (!id.containsKey(next)) {
                    id.put(next, id.size());
                    queue.add(next);
                }
                row[sym] = id.get(next);
            }
            trans.add(row);          // BFS order == id order
        }

        System.out.println("DFA States and Transitions:");
        for (var e : id.entrySet()) {
            int i = e.getValue();
            boolean fin = e.getKey().stream().anyMatch(NFA_ACCEPT::contains);
            System.out.printf("State %d = %s%s%n", i, e.getKey(), fin ? "  (final)" : "");
            System.out.println("  0 -> " + trans.get(i)[0]);
            System.out.println("  1 -> " + trans.get(i)[1]);
        }
    }
}
```

Sample Output

```
DFA States and Transitions:
State 0 = [0]
  0 -> 1
  1 -> 0
State 1 = [0, 1]  (final)
  0 -> 1
  1 -> 1
```

3. DFA State Reduction

Minimize a given DFA by identifying and merging equivalent states.

Code

```
import java.util.*;

public class DFAMinimization {
    public static void main(String[] args) {
        int[][] dfa = {{1, 2}, {1, 2}, {1, 2}};
        boolean[] fin = {false, true, true};
        int n = dfa.length, k = dfa[0].length;
```

```
        int[] group = new int[n];
        for (int i = 0; i < n; i++) group[i] = fin[i] ? 1 : 0;
        int count = (int) Arrays.stream(group).distinct().count();

        while (true) {
            Map<String, Integer> map = new HashMap<>();
            int[] next = new int[n];
            for (int i = 0; i < n; i++) {
                StringBuilder key = new StringBuilder().append(group[i]);
                for (int c = 0; c < k; c++) key.append(',').append(group[dfa[i][c]]);
                next[i] = map.computeIfAbsent(key.toString(), x -> map.size());
            }
            group = next;
            if (map.size() == count) break;   // no new split -> stable
            count = map.size();
        }

        System.out.println("Minimized DFA (" + count + " states):");
        boolean[] done = new boolean[count];
        for (int i = 0; i < n; i++) {
            if (done[group[i]]) continue;
            done[group[i]] = true;
            System.out.print("Group " + group[i] + (fin[i] ? " (final)" : ""));
            for (int c = 0; c < k; c++)
                System.out.print("  " + c + "->" + group[dfa[i][c]]);
            System.out.println();
        }
    }
}
```

Sample Output

```
Minimized DFA (2 states):
Group 0  0->1  1->1
Group 1 (final)  0->1  1->1
```

4. Intersection of Two DFAs

Given two DFAs, construct a DFA that accepts the intersection of their languages: L = L1 ∩ L2 using the product construction method.

Code

```
import java.util.*;

public class DFAIntersection {
    // DFA1: ends in 1
    static final int[][] D1 = {{0, 1}, {0, 1}};
    static final boolean[] F1 = {false, true};
    // DFA2: even number of 1s
    static final int[][] D2 = {{0, 1}, {1, 0}};
    static final boolean[] F2 = {true, false};

    static boolean accepts(String s) {
        int p = 0, q = 0;                       // product state (p, q)
        for (char ch : s.toCharArray()) {
            if (ch != '0' && ch != '1') return false;
            int c = ch - '0';
            p = D1[p][c];
            q = D2[q][c];
        }
        return F1[p] && F2[q];                  // F = F1 x F2
    }
```

```
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            System.out.print("Enter a binary string: ");
            System.out.println(accepts(sc.nextLine().trim()) ? "Accepted" : "Rejected");
        }
    }
}
```

Sample Output

```
Enter a binary string: 1001
Accepted
```

```
Enter a binary string: 0110
Rejected
```

5. PDA Simulation

Simulate a given Pushdown Automaton, including its stack operations, and determine whether a given input string is accepted or rejected.

Code

```
import java.util.*;

public class PDASimulation {
    static boolean accepts(String s, boolean allowEmpty) {
        Deque<Character> stack = new ArrayDeque<>();
        boolean readingOnes = false;            // q0 = pushing, q1 = popping

        for (char ch : s.toCharArray()) {
            if (ch == '0' && !readingOnes) {
                stack.push('X');
            } else if (ch == '1') {
                readingOnes = true;
                if (stack.isEmpty()) return false;
                stack.pop();
            } else {
                return false;                   // bad symbol or 0 after 1
            }
        }
        if (s.isEmpty()) return allowEmpty;
        return readingOnes && stack.isEmpty();
    }

    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            System.out.print("Enter a string: ");
            System.out.println(accepts(sc.nextLine().trim(), false) ? "Accepted" : "Rejected");
        }
    }
}
```

Sample Output

```
Enter a string: 000111
Accepted
```

```
Enter a string: 00111
Rejected
```